

High-Level Overview of the Perplexia AI App Structure

The Perplexia AI application is organized as a shared runtime plus assignment-specific modules. The shared runtime owns the command-line entry point, Gradio interface, chat contract, and reusable tools; Assignment 2 contains the three student implementations and their presentation configuration.

Project Structure

Directory layout in the downloaded LangGraph/code folder:

```
code/
├── run.py                → Command-line entry point
├── perplexia_ai/
│   ├── __init__.py
│   ├── app.py            → Shared Gradio UI and dynamic assignment loader
│   ├── core/
│   │   ├── __init__.py
│   │   └── chat_interface.py    → Shared ChatInterface contract
│   ├── tools/
│   │   ├── __init__.py
│   │   └── calculator.py    → Shared calculator tool
│   └── assignment_02/
│       ├── __init__.py
│       ├── app_config.py    → Titles, descriptions, and examples
│       ├── factory.py        → Maps Assignment02Mode to an implementation
│       ├── part1.py          → Query understanding starter
│       ├── part2.py          → Basic tools starter
│       └── part3.py          → Conversation memory starter
```

Core Concept: Chat Interface Design

ChatInterface: The AI Assistant Blueprint

The `ChatInterface` abstract base class defines the common contract used by every assignment implementation.

Two key methods:

- `initialize()`: Called once after the selected class is created. Use it to prepare models, prompts, tools, and other components.
- `process_message(message, chat_history)`: Called for each user message. It receives the current message and Gradio chat history, then returns the assistant response.

In **Assignment 2**, the factory can create one of three student implementations:

- **QueryUnderstandingChat** (Part 1): Classifies a query and formats the response for its category.
- **BasicToolsChat** (Part 2): Adds calculator detection and one-shot tool use.
- **MemoryChat** (Part 3): Uses the supplied chat history to handle follow-up questions.

System Flow Overview

1. Start from the project directory with `python code/run.py --assignment 2 --mode part1`. Replace `part1` with `part2` or `part3` as you progress.
2. `run.py` detects the packaged assignments, parses `--assignment` and `--mode`, and calls `create_demo()` in `app.py`.
3. `app.py` dynamically imports the Assignment 2 `factory` and `app_config` modules. The configuration supplies the UI title, description, and examples.
4. `Assignment02Mode` maps the selected mode to `QueryUnderstandingChat`, `BasicToolsChat`, or `MemoryChat`. The factory creates that class.
5. The shared app calls `initialize()` once, then constructs the Gradio chat interface.
6. For every message, Gradio calls `respond(message, history)`, which delegates to the selected class's `process_message()` method.

Current application flow

